overload of `pop_head()` that retrieves the stored value prior to modifying the list.

In comparison, `empty()` is trivial: just lock `head_mutex` and check for `head==get_tail()` (see listing 6.10). The final code for the queue is shown in listings 6.7, 6.8, 6.9, and 6.10.

Listing 6.7. A thread-safe queue with locking and waiting: internals and interface

```
template<typename T>
class threadsafe_queue
{
private:
    struct node
    {
        std::shared_ptr<T> data;
        std::unique_ptr<node> next;
    };

    std::mutex head_mutex;
    std::unique_ptr<node> head;
    std::mutex tail_mutex;
    node* tail;
    std::condition_variable data_cond;
public:
    threadsafe_queue():
        head(new node),tail(head.get())
    {}
    threadsafe_queue(const threadsafe_queue& other)=delete;
    threadsafe_queue& operator=(const threadsafe_queue&
other)=delete;

    std::shared_ptr<T> try_pop();
    bool try_pop(T& value);
    std::shared_ptr<T> wait_and_pop();
    void wait_and_pop(T& value);
    void push(T new_value);
    void empty();
};
```

Pushing new nodes onto the queue is fairly straightforward—the implementation (shown in the following listing) is close to that shown previously.

Listing 6.8. A thread-safe queue with locking and waiting: pushing new values